Energy-Efficient Deep Learning Project

Vyshnavi priya Kasarla, Madhava Narasimha Ajay Varma Penmatsa

MA5790

Abstract

As deep learning continues to scale across research and industry, the energy required to train modern models has grown dramatically. Training workloads now run on large clusters of CPUs and GPUs, often for hours or days, and the resulting energy consumption has significant implications for cost, sustainability, and operational planning. However, researchers and engineers currently have no reliable way to estimate how much energy a training job will consume before it is executed. This uncertainty leads to inefficient scheduling, unpredictable resource usage, and increased environmental and financial impact.

This project addresses this challenge by exploring how job-level information such as model configuration, resource usage patterns, and runtime characteristics can be leveraged to anticipate energy consumption in advance. By studying these relationships and developing predictive approaches, our goal is to reduce uncertainty, support energy-aware planning, and ultimately contribute toward more efficient and sustainable deep learning workflows.

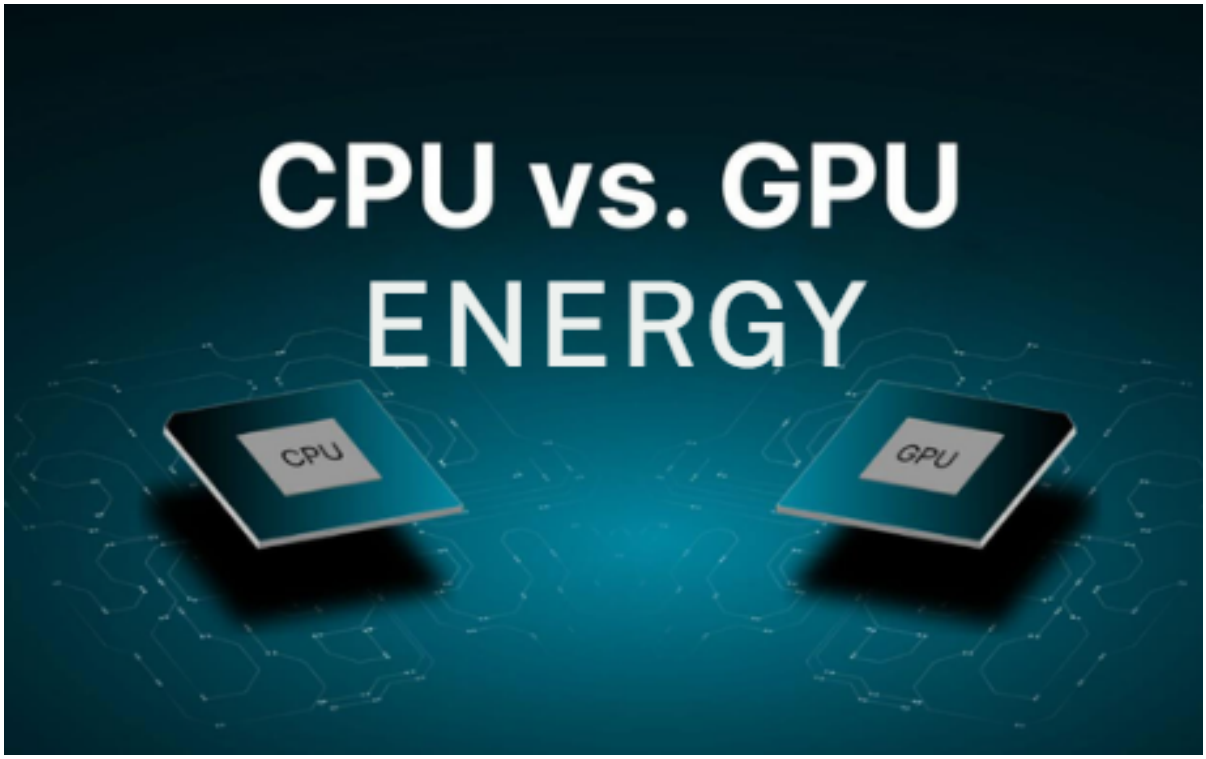


Table of Contents

1. Introduction

1.1. Background and Motivation
1.2. Data Resources
1.3. Structure of the Data

2. Pre-processing of Data

2.1. Initial Data Inspection
2.2. Timestamp Feature Engineering
2.3. Missing Value Treatment
2.4. Categorical Encoding
2.5. Near-Zero-Variance (NZV) Filtering
2.6. Train/Test Split
2.7. Creation of Two Training Copies
2.8. Box–Cox Transformation
2.9. Centering and Scaling
2.10. Spatial Sign Transformation
2.11. Target Variable Scaling
2.12. Model Tuning Preparation (Cross-Validation Setup)

3. Model Building

3.1. Linear Models
3.2. Nonlinear Models
3.3. Training Summary Table of All Models
3.4. Test Set Evaluation
3.5. Selection of the Best Model and Important Features

4. Summary

5. References

6. Code

1. Introduction

1.1 Background and Motivation

Modern deep learning workloads consume significant computational resources, often running on multi-GPU or CPU clusters for extended periods. As model architectures grow larger and training pipelines become increasingly complex, the energy usage associated with these experiments has become a serious operational concern. Yet practitioners typically lack a reliable method to estimate energy consumption before launching a job, resulting in unpredictable computational costs, inefficient scheduling decisions, and difficulties in resource planning. These uncertainties also contribute to unnecessary financial overhead and increased environmental impact.

Accurately predicting energy usage *before* a deep learning job executes would allow organizations to manage compute resources more responsibly and design experiments more efficiently. This motivates the need for predictive modeling approaches that use workload characteristics such as architecture details, batch-level CPU energy, shapes and depths, runtime behavior, and scheduling patterns to estimate total energy consumption. By modeling these relationships, the project aims to support more informed planning, budgeting, and sustainable AI practices.

1.2 Data Resources

The data used in this project comes from the **BUTTER-E: Energy Consumption for Deep Learning Workloads** dataset, published by the National Renewable Energy Laboratory (NREL).

Dataset Link: https://data.openei.org/submissions/5991

Dataset Size Used in This Study:

- **29,647 observations**
- **33 predictors**

1.3 Structure of the Data

The dataset consists of a combination of continuous and categorical predictors derived from deep learning workload logs. These variables describe runtime characteristics, batch-level CPU energy usage, system overhead, model configuration, and scheduling information. After timestamp feature engineering and before dummy encoding, the dataset contains **33 predictors** and **29,647 samples**.

A summary of the variable names and their descriptions is provided below.

Continuous Predictors (20 variables)

These variables are stored as numeric values and represent measurable quantities related to runtime, energy, model configuration, and overhead statistics.

- **experiment_id, slurm_job_id** – Numeric identifiers for each job and scheduling instance.
- **size, depth** – Model architecture parameters describing layer size and network depth.
- **learning_rate** – Training learning rate used in the workload.
- **batch_size** – Number of samples processed per training batch.
- **is_gpu** – Indicator (0/1) showing whether the job used GPU resources.
- **run_time** – Total runtime of the workload in seconds.
- **start_time_int, start_offset** – Engineered timestamp-based numeric variables capturing starting time conditions.
- **filter_3_stddevs** – Statistical measure describing variability in a specific filter parameter.
- **power, std_power** – Average and standard deviation of power usage.
- **std_energy** – Standard deviation of energy consumption.
- **num_reps** – Number of repetitions or iterations performed.
- **non_overhead_energy, energy_overhead** – Energy consumed by non-overhead and overhead components of the job.
- **runtime_overhead, non_overhead_runtime** – Time spent on overhead vs. actual computation.

These continuous features provide information about hardware utilization, workload structure, and energy dynamics.

Categorical Predictors (13 variables)

These variables represent descriptive labels, identifiers, and configuration choices. They are later converted into dummy variables for modeling.

- **run_id, run_id.1** – Unique string identifiers for runs, often used for grouping or logging.
- **start_time, update_time** – Raw timestamp variables representing when the job started and last updated.
- **node** – Name of the compute node used for execution.
- **shape** – Describes the model's shape configuration (e.g., square, trapezoid).
- **dataset** – Dataset used for training (e.g., CIFAR, MNIST).
- **optimizer** – Optimization method used (e.g., SGD, Adam).
- **batch** – High-level batching category (such as small/medium/large).

- **filter_1, filter_2, filter_3, filter** – Categorical descriptors of different filter settings or model configuration options.

These variables define structural, configuration-based, and scheduling information about each workload.

Target Variable

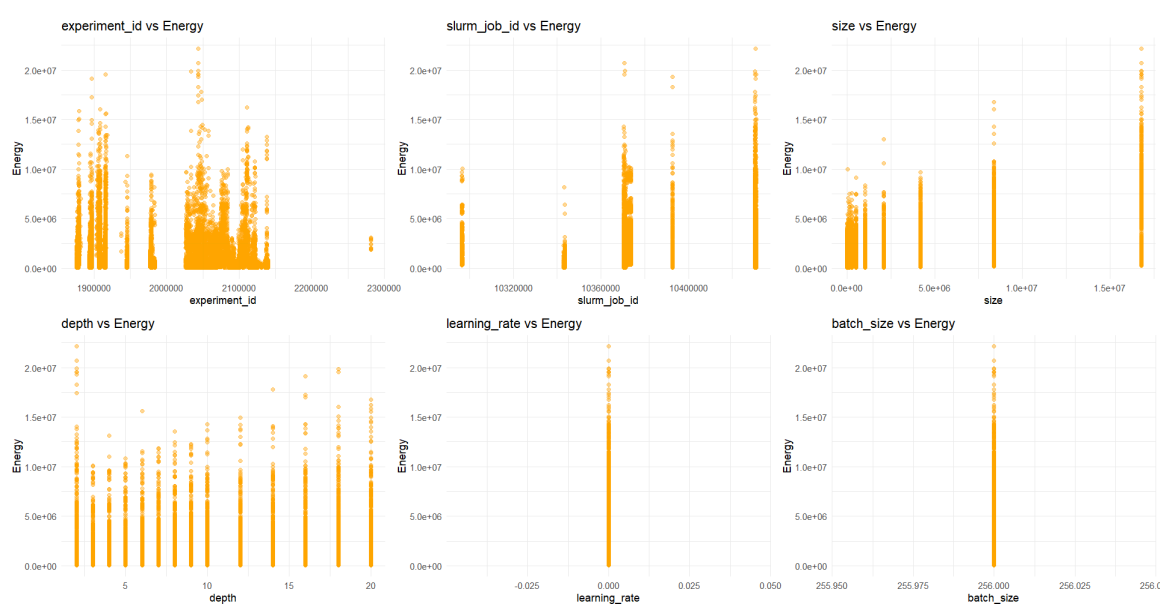
- **energy** – Total energy consumption of the workload (in joules). This is the variable we aim to predict using regression models.

2. Pre-processing of Data

2.1 Initial Data Inspection

The preprocessing stage began with an initial inspection of the raw dataset, which contained 33 variables describing model configuration, execution environment, runtime statistics, and energy measurements. This inspection involved printing variable types, examining sample entries, and quantifying the distribution of numeric and categorical fields. During this phase, exploratory visualizations were generated such as structural summaries and variable-type plots to confirm that the dataset contained a mix of continuous and categorical predictors. Understanding this structure was crucial for selecting appropriate cleaning and transformation methods used later in the workflow.

This is the plots for the predictors vs response variables for few predictors:

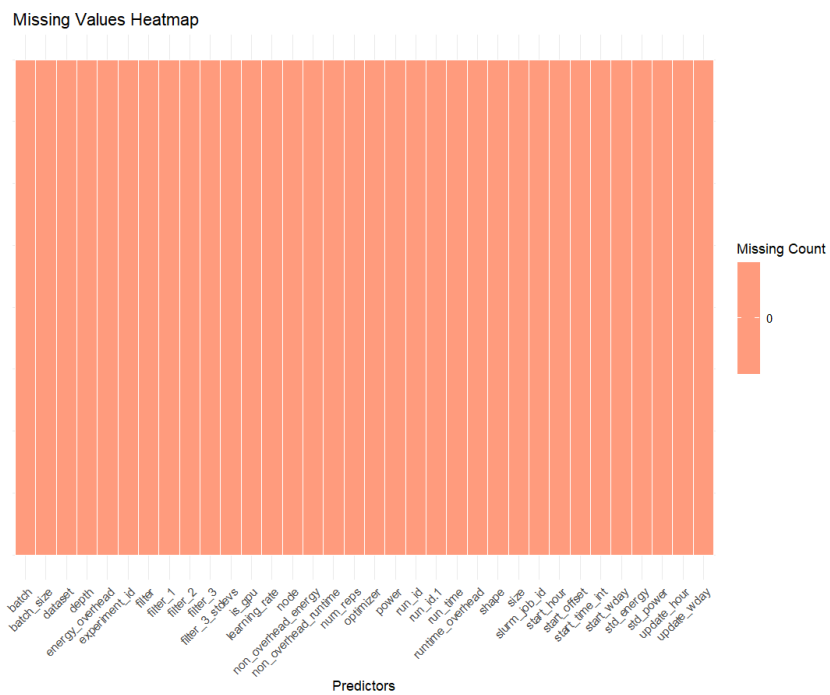


2.2 Timestamp Feature Engineering

The dataset included raw timestamp fields (start_time and update_time), stored as character variables. These were first converted into POSIX-compliant datetime objects to ensure proper temporal interpretation. From each timestamp, additional engineered variables were created, including hour-of-day and day-of-week features, which capture temporal scheduling patterns associated with deep learning jobs. These engineered predictors were appended to the dataset, increasing the feature richness and enabling the model to learn from temporal execution behaviors. Corresponding distribution plots and summary visuals are included to highlight how these time-based features vary across the dataset.

2.3 Missing Value Treatment

The dataset was examined to check for missing values across all predictors. Both numeric and categorical variables were inspected, and a missingness heatmap was generated to visually confirm the presence or absence of gaps. The analysis showed that there were **zero missing values** in all predictors, meaning no imputation was required. As a result, the dataset was already complete, and no additional steps were needed to address missing data.



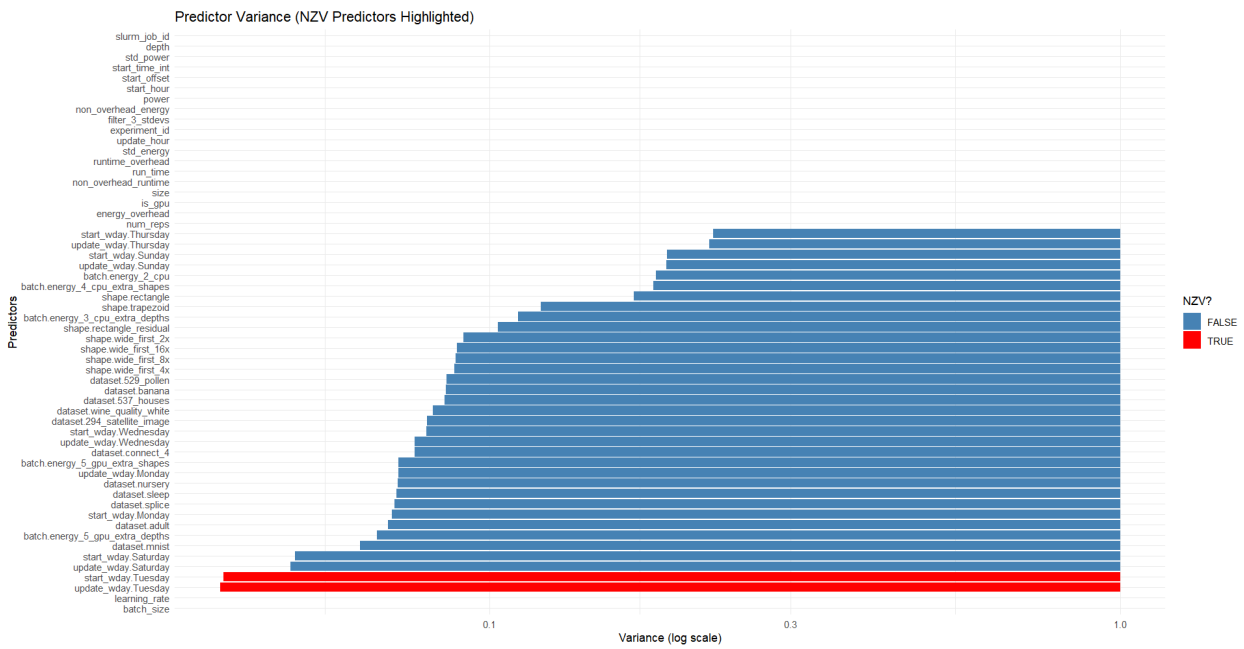
2.4 Categorical Encoding

Categorical variables were prepared for modeling through dummy encoding. Prior to encoding, non-informative ID-like variables including run_id, run_id.1, and node were removed because

they provide no predictive signal and could introduce noise. The remaining categorical variables were converted to factor form, and any variables with only one level were discarded. Using dummyVars(), each categorical predictor was expanded into binary dummy columns, increasing the total predictor count from 33 to 56. This expansion allowed linear and nonlinear regression models to properly interpret categorical structure. Diagnostic printouts and structural summaries illustrate the expanded design matrix.

2.5 Near-Zero-Variance (NZV) Filtering

To eliminate uninformative predictors, a near-zero-variance (NZV) analysis was conducted. Predictors with extremely low variability indicating they contain little useful information were removed. This step reduced the predictor count from 56 to 52. A variance visualization is included in the report, where predictors flagged as NZV are highlighted in red, making it clear why these features lacked predictive value. Additional histogram panels demonstrate the narrow distributions that caused their removal.



2.6 Train/Test Split

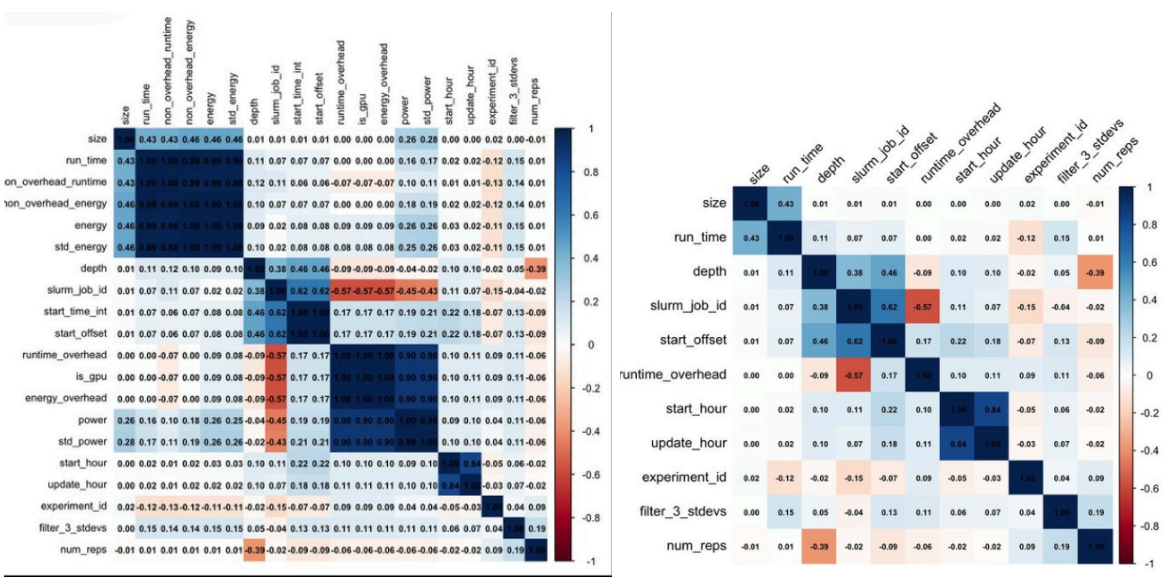
Following preprocessing, the dataset was partitioned using simple random sampling, with 80% of the data allocated to the training set and 20% reserved for independent testing. This approach ensures that the training distribution reflects the overall dataset and provides an unbiased evaluation of model performance. The exact sample sizes for both partitions were reported, confirming a balanced and representative split.

```
> cat("Train size:", nrow(train_data), "\n")
Train size: 29647
> cat("Test size :", nrow(test_data), "\n")
Test size : 7408
```

2.7 Creation of Two Training Copies

To address the different requirements of the modeling algorithms, two distinct versions of the training dataset were constructed after NZV filtering. The first version removed highly correlated predictors using a correlation threshold of 0.85, producing a cleaner feature set for models sensitive to multicollinearity such as Linear Regression, Neural Networks, and MARS. The second version retained all predictors because models like PLS, Ridge, Lasso, Elastic Net, SVM, and KNN are inherently robust to collinearity through latent components, regularization, or distance-based learning. Correlation plot shows the before and after correlation removal for predictors. The before removal plot included in this section illustrates the strong predictor clusters and shows exactly which features were removed in the collinearity-filtered dataset.

```
Copy 1 - Predictors removed due to correlation > 0.85:
> print(high_corr_names_c1)
 [1] "std_power"              "power"                  "is_gpu"
 [4] "runtime_overhead"       "start_wday.Thursday"    "start_offset"
 [7] "update_wday.Sunday"     "std_energy"             "non_overhead_runtime"
[10] "non_overhead_energy"    "update_wday.Monday"     "start_wday.Wednesday"
[13] "update_wday.Saturday"
>
```

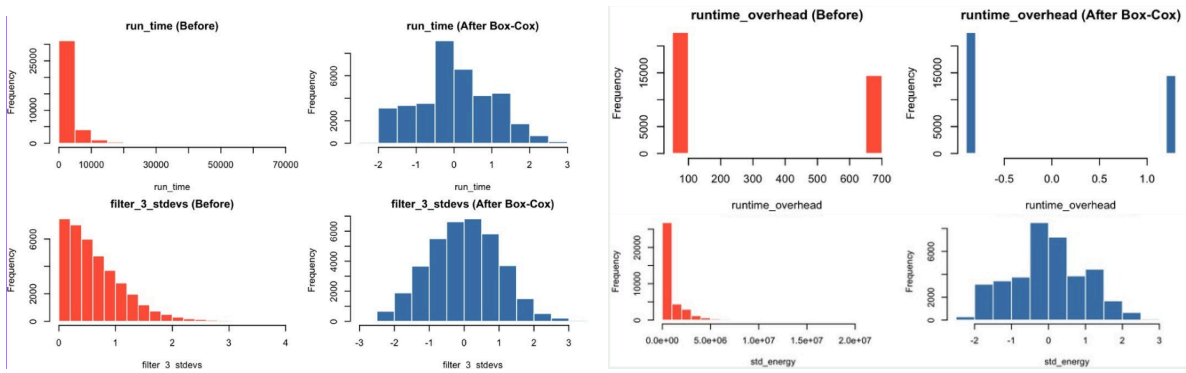


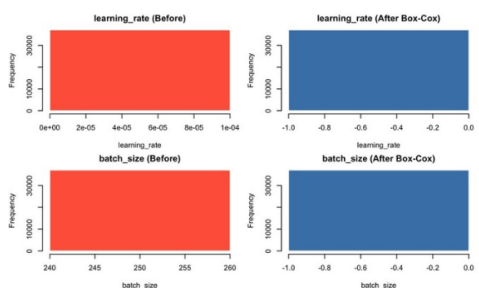
Before and After correlated predictors removal

2.7.1 Box–Cox Transformation

After defining the two dataset copies, the Box–Cox transformation was applied to continuous predictors to correct skewness and stabilize variance. Many runtime and energy-related variables exhibited right-skewed behavior, which can negatively affect regression models and gradient-based algorithms. Box–Cox smoothing makes distributions more symmetric, improving model fit and numerical performance.

The before-and-after histograms included here demonstrate how the transformation reduces skew and normalizes the shapes of key predictors.





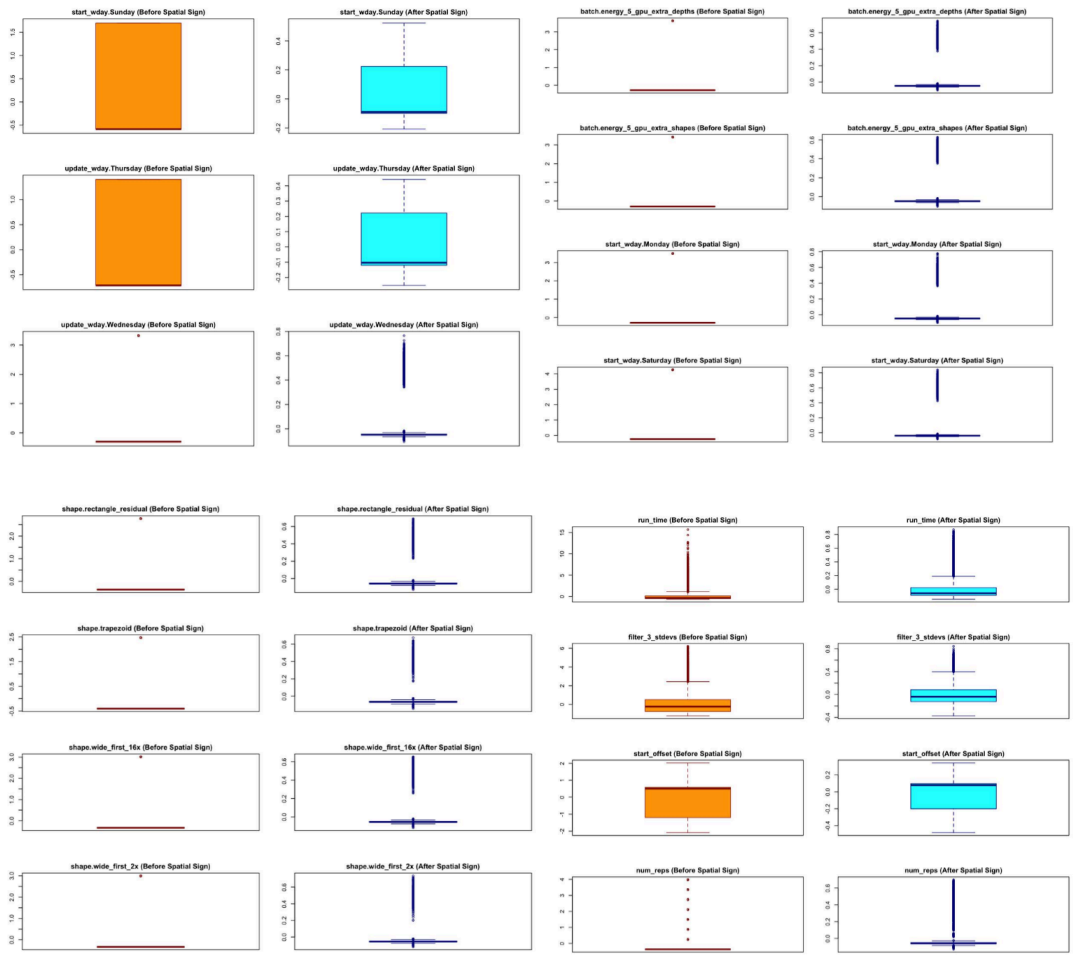
2.7.2 Centering and Scaling

Following Box–Cox, all predictors were centered and scaled to ensure that each variable contributed on a comparable numeric scale. This normalization is critical for algorithms that rely on geometric distances (KNN), kernel functions (SVM), or gradient-based optimization (Neural Networks), as well as for regularized models like Ridge, Lasso, and Elastic Net. Standardization prevents any single predictor from dominating due to its magnitude.

2.7.3 Spatial Sign Transformation

To further stabilize model behavior, especially in high-dimensional predictor spaces, the spatial sign transformation was applied. This technique projects each observation onto the surface of a unit hypersphere, effectively normalizing vector magnitudes while preserving directional

information. By compressing extreme values, the spatial sign transformation reduces the influence of outliers and improves stability for distance-based and gradient-sensitive models. *Visualizations in this section highlight how extreme magnitudes shrink after applying spatial sign, resulting in a more uniform and outlier-resistant predictor space.*



2.8 Target Variable Scaling

Finally, the target variable (energy) was standardized by mean-centering and scaling using only the training set statistics. This avoids information leakage and ensures numerical stability during model optimization. The scaled target was used internally by the models, but all predictions on the test set were inverse-transformed back to the original energy units for accurate interpretation of RMSE, MAE, and R^2.

2.9 Model Tuning and Resampling Strategy

All models were trained using a **10-fold cross-validation** resampling design (trainControl(method = "cv", number = 10)), which partitions the training data into ten subsets,

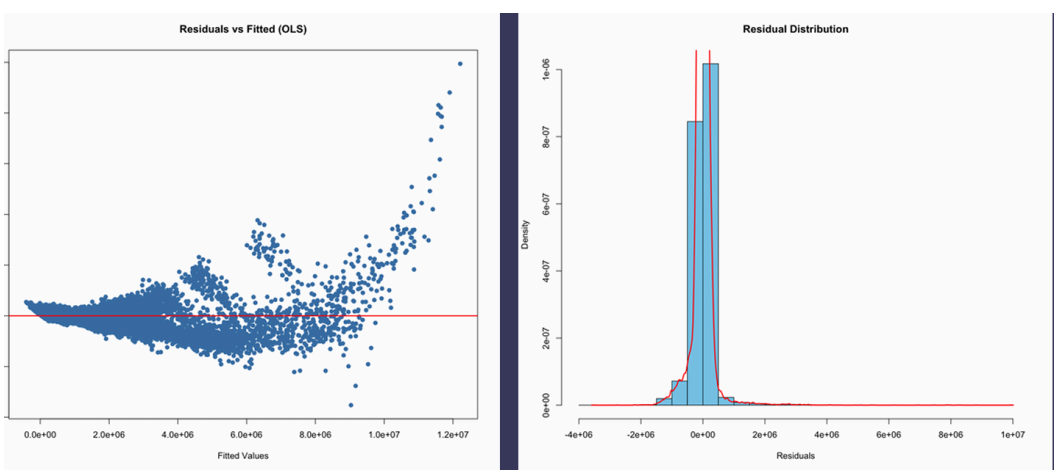
repeatedly training on nine folds and validating on the remaining one. This method provides robust estimates of training RMSE, MAE, and R^2 while reducing variance and guarding against overfitting. Each model's hyperparameters such as penalty strengths for ridge/lasso, component counts for PLS, decay and hidden units for neural networks, knot counts for MARS, and kernel parameters for SVM were selected by minimizing cross-validated RMSE.

3. Model Building

To evaluate how effectively workload characteristics can predict total energy consumption, a range of linear and nonlinear regression models were trained using the preprocessed datasets. All models were implemented using the caret framework in R, which provides a unified interface for model fitting, hyperparameter tuning, and cross-validation. Consistent with the preprocessing design, two predictor matrices were used: the **Copy 1 dataset** with collinearity removed for models that are sensitive to correlated predictors, and the **Copy 2 dataset** with all predictors retained for models that either regularize or naturally handle multicollinearity. Each model was trained using **10-fold cross-validation**, ensuring stable and unbiased performance estimates on the training data.

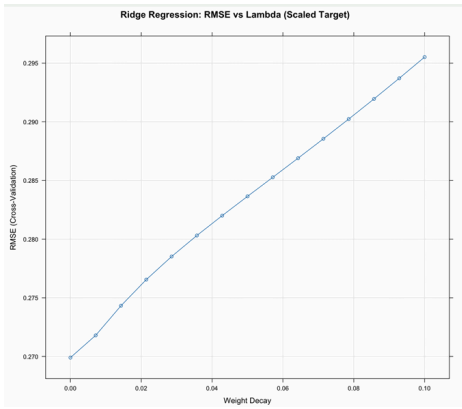
3.1 Linear Models

Linear regression (OLS) served as a baseline model due to its interpretability and simplicity. Using the Copy 1 predictor set, the model was trained on standardized predictors and a centered-and-scaled target variable. However, the residual diagnostics indicated violations of linearity and homoscedasticity, which suggested that the relationship between predictors and energy consumption is inherently nonlinear. Nonetheless, OLS provides a useful reference point for comparing more flexible models.

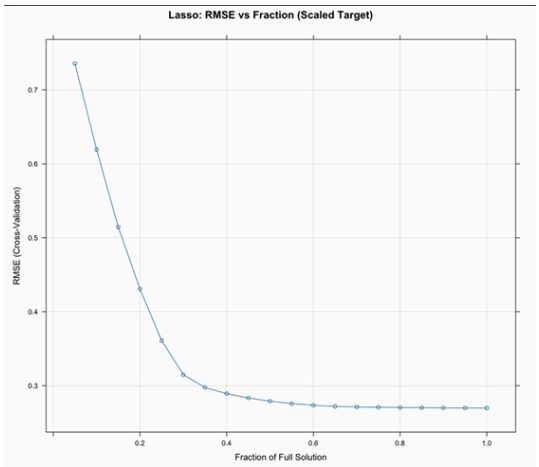


Penalized linear models including Ridge Regression, Lasso, and Elastic Net were trained using the Copy 2 dataset, where correlated predictors remained intact.

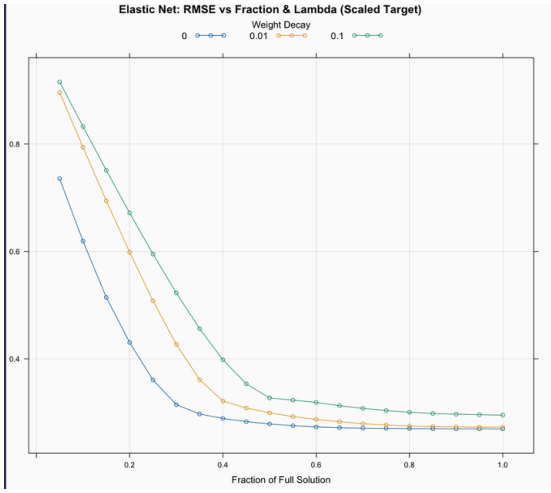
Ridge Regression was trained using the Copy 2 dataset, retaining correlated predictors to leverage the stabilizing effect of L2 regularization. This model reduces coefficient variance and alleviates multicollinearity but does not perform feature selection.



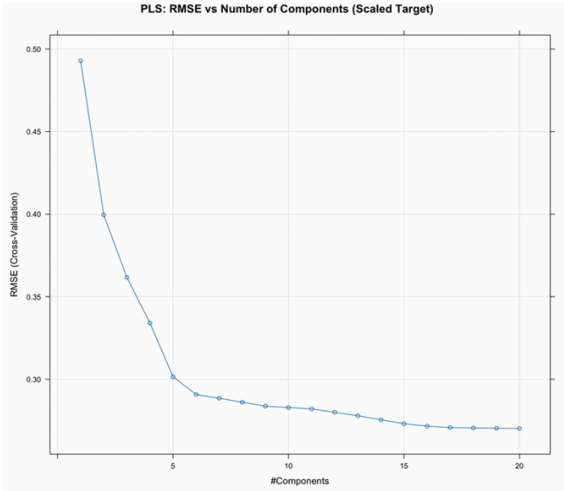
Lasso (L1 regularization) encourages sparsity by shrinking some coefficients to zero, effectively performing variable selection. It was tuned via cross-validation on Copy 2, where correlated predictors were preserved for the model to choose among.



Elastic Net combines both L1 and L2 penalties, balancing Ridge's stability with Lasso's variable selection. It was trained on Copy 2 and tuned over a grid of λ and fraction values to find the optimal compromise.



PLS regression extracts latent components that maximize covariance between predictors and the response. Although it handles correlated predictors well, its linear component structure limits performance in nonlinear settings.

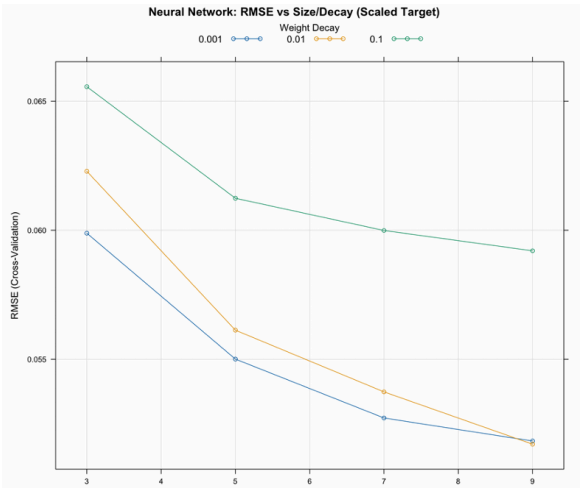


3.2 Nonlinear Models

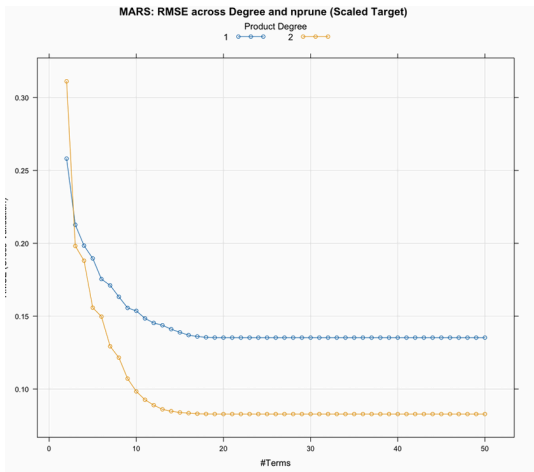
Given the evident nonlinear relationships in the data, several nonlinear regression models were trained to capture complex interactions among predictors.

A **model-averaged neural network** with a single hidden layer was trained using the Copy 1 dataset. Hyperparameters hidden layer size (3–9 units) and weight decay (0.001–0.1) were tuned

using 10-fold cross-validation. Because neural networks are sensitive to predictor scale and magnitude outliers, the Copy 1 preprocessing pipeline (Box–Cox → Center/Scale → Spatial Sign) significantly improved the model's numerical stability and convergence. The neural network achieved one of the lowest RMSE values, demonstrating strong ability to capture nonlinear workload–energy relationships.

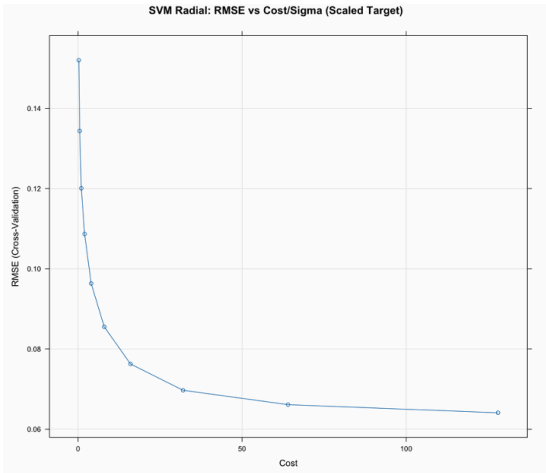


MARS was trained on Copy 1 to automatically detect nonlinearities and interactions by fitting piecewise linear spline functions. Model tuning varied both the spline degree and the number of pruned terms (nprune). MARS substantially outperformed linear models, confirming the presence of threshold effects and nonlinear scaling behavior in deep-learning workloads.

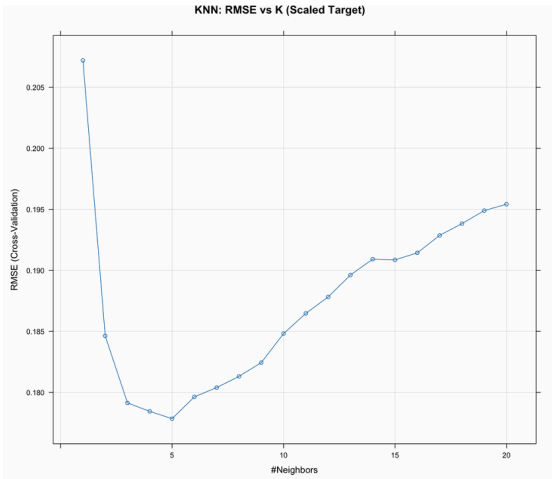


SVM with an RBF kernel was trained using Copy 2, where correlated predictors were retained to

allow the kernel function to leverage their joint structure. SVM relies heavily on standardized predictors, making the preprocessing pipeline (center + scale + spatial-sign) essential. The model was tuned over combinations of **C (cost)** and **sigma**, resulting in strong predictive accuracy and among the best RMSE values.



KNN was also trained on Copy 2 due to its reliance on distance-based comparisons, which require consistent feature scales. Spatial-sign transformation reduced the influence of high-magnitude outliers, ensuring smoother neighborhood boundaries. A tuning grid over k = 1–20 was evaluated using 10-fold CV. KNN performed competitively, particularly in regions where energy relates smoothly to workload parameters.



3.3 Training Summary Table of All Models

Models	Training RMSE	Training MAE	Training R2
OLS	0235	0.125	0.944
Ridge	0.269	0.124	0.927
Lasso	0.269	0.123	0.926
ElasticNet	0.269	0.125	0.926
PLS	0.270	0.124	0.927
Neural network	**0.051**	**0.024**	**0.997**
MARS	0.084	0.042	0.993
SVM	**0.064**	**0.046**	**0.996**
KNN	0.157	0.072	0.976

3.4 Test Set Evaluation

The two best-performing models **Neural Network (avNNet)** and **SVM with RBF kernel** were evaluated on the untouched test set. Predictions were first generated in the scaled target space and then inverse-transformed back to the original energy units for proper interpretation. Test-set performance was assessed using RMSE, MAE, and R^2, and both models demonstrated strong generalization, with the neural network and SVM achieving the lowest error values among all candidates.

Models	Test RMSE	Test MAE	Test R2
Neural Network	0.0498	0.0233	0.9974
SVM	0.0591	0.0461	0.9965

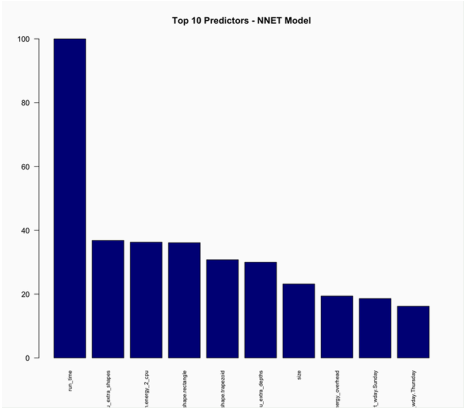
Models were ranked based on test-set RMSE, where lower values indicate superior predictive accuracy**.** These two models captured the complex nonlinear dependencies between job parameters and energy consumption more effectively than all other approaches. Their predictions generalized well to unseen data, confirming their suitability as finalists for energy prediction.

3.5 Selection of the Best Model and Important features

Between the two top-performing nonlinear models **SVM-Radial** and the **Neural Network (avNNet)** the **Neural Network** emerged as the superior final model. Although both models produced strong results, the neural network achieved **lower test RMSE and higher R^2**, indicating better generalization to unseen workloads. Its architecture allows it to capture layered nonlinear interactions that the SVM, even with an RBF kernel, cannot fully represent.

Model interpretability was enhanced by extracting the top predictors from the neural network, which revealed **run_time** as the dominant driver of energy consumption, followed by **batch-level CPU energy features** (e.g., *batch.energy_4_cpu_extra_shapes*, *batch.energy_2_cpu*, *batch.energy_3_cpu_extra_depths*), **geometric architecture descriptors** (*shape.rectangle*, *shape.trapezoid*), and **operational factors** such as workload size and weekday scheduling effects. These insights confirm that energy usage is influenced by a combination of execution duration, CPU-intensive batch operations, model shape configurations, and system-level scheduling patterns complex relationships that the neural network captures more effectively than the SVM.

```
Top 10 Predictors for NNet:
> print(top10_nn)
                                  Overall                       Predictor
run_time                         100.00000                        run_time
batch.energy_4_cpu_extra_shapes   36.83168 batch.energy_4_cpu_extra_shapes
batch.energy_2_cpu                36.27281              batch.energy_2_cpu
shape.rectangle                   36.10074                 shape.rectangle
shape.trapezoid                   30.76305                 shape.trapezoid
batch.energy_3_cpu_extra_depths   29.99047 batch.energy_3_cpu_extra_depths
size                              23.17193                            size
energy_overhead                   19.40126                 energy_overhead
start_wday.Sunday                 18.61171               start_wday.Sunday
update_wday.Thursday              16.20678            update_wday.Thursday
```



4. Summary

This project focused on predicting the energy consumption of deep learning workloads using information such as runtime, CPU energy measurements, model shape, and scheduling details. After carefully preparing the data through cleaning, encoding, feature engineering, and scaling, several linear and nonlinear regression models were trained and compared. Nonlinear models performed much better than linear ones, and the Neural Network (avNNet) emerged as the best overall model with the highest accuracy on the test set. The most important predictors included runtime, batch-level CPU energy values, and shape-related model features, showing that energy usage depends heavily on how long the job runs, how much CPU work it performs, and the structure of the workload itself.

References

[1] Tripp, C., Perr-Sauer, J., Bensen, E., Gafur, J., Nag, A., & Purkayastha, A. (2022). *BUTTER-E - Energy Consumption Data for the BUTTER Empirical Deep Learning Dataset*. [Data set]. Open Energy Data Initiative (OEDI). National Renewable Energy Laboratory. https://doi.org/10.25984/2329316

[2] Strubell, E., Ganesh, A., & McCallum, A. (2019). *Energy and policy considerations for deep learning in NLP*. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 3645–3650. https://arxiv.org/abs/1906.02243

[3] Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L. M., Rothchild, D., So, D., Texier, M., & Dean, J. (2021). *Carbon emissions and large neural network training*. arXiv:2104.10350. https://arxiv.org/abs/2104.10350

[4] Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., & Pineau, J. (2020). *Towards the systematic reporting of the energy and carbon footprints of machine learning*. Journal of Machine Learning Research, 21(248), 1–43. http://www.jmlr.org/papers/v21/20-312.html

[5] Friedman, J. H. (1991). *Multivariate adaptive regression splines*. Annals of Statistics, 19(1), 1–67. https://projecteuclid.org/journals/annals-of-statistics/volume-19/issue-1/Multivariate-Adaptive-Regression-Splines/10.1214/aos/1176347963.full

[6] Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). *A training algorithm for optimal margin classifiers*. Proceedings of the Fifth Annual Workshop on Computational Learning Theory, 144–152. https://dl.acm.org/doi/10.5555/130385.130401

R CODE:

```
########################### Project ####################################
## Initializing libraries
library(caret)
library(corrplot)
library(e1071)
library(ggplot2)
library(reshape2)
library(earth)
###########################  STEP 0: LOAD DATA  #######################

Original_dataset<-read.csv("~/Desktop/PredictiveModeling/runs_with_standardized_energy.csv"
)
dim(Original_dataset)

##################################################################################
# Dataset Overview & EDA (till timestamp dataset update)
##################################################################################

print("Preview of the dataset (first 5 rows, first 33 columns):")
Original_dataset[1:5, 1:33]

print("All predictors:")
names(Original_dataset)
cont_vars <- names(continuous_dataset)

plot_list <- lapply(cont_vars, function(v) {
  ggplot(updated_dataset, aes_string(x = v, y = "energy")) +
    geom_point(alpha = 0.4, color = "orange") +
    theme_minimal() +
    labs(title = paste(v, "vs Energy"), x = v, y = "Energy")
})

# Show first 6 plots in a grid
grid.arrange(grobs = plot_list[1:6], ncol = 3)

# Copy for manipulation
copy_dataset <- Original_dataset
```

```
######### Predictors Overview (Original) #########

# Continuous predictors
dataset_numeric <- sapply(copy_dataset, is.numeric)
continuous_dataset <- copy_dataset[, dataset_numeric]
print("Continuous Predictors:")
names(continuous_dataset)

count_continuous <- sum(dataset_numeric)
cat("Number of Continuous Predictors:", count_continuous, "\n")

# Categorical predictors
categorical_dataset <- copy_dataset[, sapply(copy_dataset, function(x) is.factor(x) || is.character(x))]
print("Categorical Predictors:")
names(categorical_dataset)

count_categorical <- sum(sapply(copy_dataset, function(x) is.factor(x) || is.character(x)))
cat("Number of Categorical Predictors:", count_categorical, "\n")

######### Timestamp Column Transformation #########

# Convert timestamps
copy_dataset$start_time <- as.POSIXct(copy_dataset$start_time, format = "%Y-%m-%d %H:%M:%OS", tz = "UTC")
copy_dataset$update_time <- as.POSIXct(copy_dataset$update_time, format = "%Y-%m-%d %H:%M:%OS", tz = "UTC")

# Create engineered time features
start_hour <- as.numeric(format(copy_dataset$start_time, "%H"))
start_wday <- as.factor(weekdays(copy_dataset$start_time))
update_hour <- as.numeric(format(copy_dataset$update_time, "%H"))
update_wday <- as.factor(weekdays(copy_dataset$update_time))

# Concatenate engineered columns with original dataset
updated_dataset <- cbind(copy_dataset, start_hour, start_wday, update_hour, update_wday)

cat("Structure of updated_dataset after timestamp engineering:\n")
str(updated_dataset)
```

```
head(updated_dataset)

##### UPDATED DATASET PREDICTOR OVERVIEW ####

# Numeric columns
updated_dataset_numeric <- sapply(updated_dataset, is.numeric)
continuous_updated_dataset <- updated_dataset[, updated_dataset_numeric]
cat("Updated Continuous Predictors:\n")
names(continuous_updated_dataset)
cat("Number of updated continuous predictors:", sum(updated_dataset_numeric), "\n")

# Categorical columns
categorical_updated_dataset <- updated_dataset[, sapply(updated_dataset, function(x) is.factor(x) || is.character(x))]
cat("Updated Categorical Predictors:\n")
names(categorical_updated_dataset)
cat("Number of updated categorical predictors:",
    sum(sapply(updated_dataset, function(x) is.factor(x) || is.character(x))), "\n")

############################  STEP 2: MISSING ############################
# (Missing values handled BEFORE dummy variables)

# Separate predictors and response
predictors_raw <- updated_dataset[, !(names(updated_dataset) == "energy")]
response       <- updated_dataset$energy

# Split numeric and categorical
num_part <- predictors_raw[, sapply(predictors_raw, is.numeric), drop = FALSE]
cat_part <- predictors_raw[, sapply(predictors_raw, function(x) is.factor(x) || is.character(x)), drop = FALSE]

# Check missing before imputation
missing_before_num <- sum(is.na(num_part))
cat("Total missing values in numeric predictors BEFORE imputation:", missing_before_num, "\n")

# KNN imputation on numeric variables only
preprocess_missing <- preProcess(
  num_part,
  method = c("knnImpute")
```

```
)

num_imputed <- predict(preprocess_missing, num_part)

missing_after_num <- sum(is.na(num_imputed))
cat("Total missing values in numeric predictors AFTER imputation:", missing_after_num, "\n")

# Recombine numeric (imputed) + categorical + response
dataset_after_missing <- data.frame(
  num_imputed,
  cat_part,
  energy = response
)

#=======  VISUALIZATION: MISSING VALUES (ALL PREDICTORS)  =======#

missing_counts  <-  colSums(is.na(dataset_after_missing[,  !(names(dataset_after_missing)  ==
"energy")]))

missing_df <- data.frame(
  Predictor = names(missing_counts),
  Missing   = missing_counts
)

ggplot(missing_df, aes(x = Predictor, y = 1, fill = Missing)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "white", high = "red") +
  theme_minimal() +
  theme(
    axis.title.y = element_blank(),
    axis.text.y  = element_blank(),
    axis.ticks.y = element_blank(),
    axis.text.x  = element_text(angle = 45, hjust = 1)
  ) +
  labs(
    title = "Missing Values Heatmap",
    x     = "Predictors",
    fill  = "Missing Count"
  )
```

```
######################## STEP 3: DUMMY VARIABLES ####################
# Separate predictors and response
predictors_mv <- dataset_after_missing[, !(names(dataset_after_missing) == "energy")]
response_mv   <- dataset_after_missing$energy

# Identify categorical and numeric predictors
cat_vars <- predictors_mv[
  , sapply(predictors_mv, function(x) is.factor(x) || is.character(x)),
  drop = FALSE
]

num_vars <- predictors_mv[
  , sapply(predictors_mv, is.numeric),
  drop = FALSE
]

# Remove ID-like columns from categoricals
id_like <- c("run_id", "run_id.1", "node")
cat_vars <- cat_vars[
  , !(names(cat_vars) %in% id_like),
  drop = FALSE
]

# Ensure all categorical vars are factors
for (c in names(cat_vars)) {
  cat_vars[[c]] <- as.factor(cat_vars[[c]])
}

# Remove single-level factors
cat_vars <- cat_vars[
  , sapply(cat_vars, function(x) nlevels(x) > 1),
  drop = FALSE
]

# Dummy Encoding
if (ncol(cat_vars) > 0) {
  dummy_model <- dummyVars(" ~ .", data = cat_vars, fullRank = TRUE)
  dummy_data  <- as.data.frame(predict(dummy_model, newdata = cat_vars))
} else {
```

```
  dummy_data <- NULL
}

# Combine numeric + dummy predictors
if (!is.null(dummy_data)) {
  predictors_encoded <- cbind(num_vars, dummy_data)
} else {
  predictors_encoded <- num_vars
}

final_dataset <- data.frame(predictors_encoded, energy = response_mv)
cat("Dummy encoding completed. Total predictors now:", ncol(predictors_encoded), "\n")

########################  STEP 4: NZV & VISUALS  #######################
# Identify NZV predictors
nzv_info <- nearZeroVar(
  final_dataset[, !(names(final_dataset) == "energy")],
  saveMetrics = TRUE
)

cat("NZV summary:\n")
print(nzv_info)

nzv_to_remove <- rownames(nzv_info[nzv_info$nzv == TRUE, ])

cat("Number of NZV predictors removed:", length(nzv_to_remove), "\n")
cat("Removed NZV predictors:\n")
print(nzv_to_remove)

# Remove NZV predictors
dataset_after_nzv <- final_dataset[
  , !(names(final_dataset) %in% nzv_to_remove),
  drop = FALSE
]

cat("Dimensions after NZV filtering:", dim(dataset_after_nzv), "\n")

#=======   VISUALIZATION: VARIANCE (NZV INSPECTION)  =======#

var_values <- sapply(
```

```
  final_dataset[, !(names(final_dataset) == "energy")],
  function(x) var(x, na.rm = TRUE)
)

var_df <- data.frame(
  Predictor = names(var_values),
  Variance  = var_values,
  NZV       = names(var_values) %in% nzv_to_remove
)

ggplot(var_df, aes(x = reorder(Predictor, Variance), y = Variance, fill = NZV)) +
  geom_bar(stat = "identity") +
  scale_y_log10() +
  coord_flip() +
  scale_fill_manual(values = c("FALSE" = "steelblue", "TRUE" = "red")) +
  labs(
    title = "Predictor Variance (NZV Predictors Highlighted)",
    x     = "Predictors",
    y     = "Variance (log scale)",
    fill  = "NZV?"
  ) +
  theme_minimal()

#=====  VISUALIZATION: DISTRIBUTIONS OF NZV-REMOVED VARIABLES  =====#

before_cols <- names(final_dataset)[!(names(final_dataset) == "energy")]
after_cols  <- names(dataset_after_nzv)[!(names(dataset_after_nzv) == "energy")]

removed_cols <- setdiff(before_cols, after_cols)

cat("NZV Predictors Removed (Visual check):\n")
print(removed_cols)

if (length(removed_cols) > 0) {
  nzv_removed_df <- final_dataset[, removed_cols, drop = FALSE]
  nzv_long <- melt(nzv_removed_df)

  ggplot(nzv_long, aes(x = value)) +
    geom_histogram(bins = 30, fill = "red", color = "black") +
    facet_wrap(~variable, scales = "free", ncol = 2) +
```

```
    theme_minimal() +
    labs(
      title = "Distributions of Predictors Removed by NZV",
      x     = "Value",
      y     = "Frequency"
    )
} else {
  cat("No NZV variables to visualize.\n")
}

###################  STEP 5: TRAIN / TEST SPLIT  #######################
set.seed(2025)

predictors_nzv <- dataset_after_nzv[, !(names(dataset_after_nzv) == "energy")]
response_nzv   <- dataset_after_nzv$energy

train_index <- createDataPartition(response_nzv, p = 0.8, list = FALSE)

train_data <- dataset_after_nzv[train_index, ]
test_data  <- dataset_after_nzv[-train_index, ]

cat("Train size:", nrow(train_data), "\n")
cat("Test size :", nrow(test_data), "\n")

X_train_full <- train_data[, !(names(train_data) == "energy")]
y_train      <- train_data$energy

X_test_full  <- test_data[, !(names(test_data) == "energy")]
y_test       <- test_data$energy

#############  STEP 6: TRAINING COPIES & TRANSFORMATIONS  ##############

#----------- Copy 1: WITH Collinearity cutoff (0.85), then Box-Cox + Spatial Sign ----#

X_train_c1 <- X_train_full

# Correlation & collinearity cutoff on training only
cor_train_c1 <- cor(X_train_c1, use = "pairwise.complete.obs")
high_corr_idx_c1   <- findCorrelation(cor_train_c1, cutoff = 0.85)
high_corr_names_c1 <- colnames(X_train_c1)[high_corr_idx_c1]
```

```
cat("Copy 1 - Predictors removed due to correlation > 0.85:\n")
print(high_corr_names_c1)

# Remove from training
if (length(high_corr_idx_c1) > 0) {
  X_train_c1 <- X_train_c1[, -high_corr_idx_c1, drop = FALSE]
}

# Box-Cox + center + scale on training (Copy 1)
preprocess_c1 <- preProcess(
  X_train_c1,
  method = c("BoxCox", "center", "scale")
)

X_train_c1_bc <- predict(preprocess_c1, X_train_c1)

# Spatial Sign (Copy 1)
X_train_c1_final <- spatialSign(X_train_c1_bc)

# Prepare corresponding test predictors for Copy 1
X_test_c1 <- X_test_full[, colnames(X_train_c1), drop = FALSE]
X_test_c1_bc <- predict(preprocess_c1, X_test_c1)
X_test_c1_final <- spatialSign(X_test_c1_bc)


#----------- Copy 2: NO Collinearity cutoff; Box-Cox + Spatial Sign ----#

X_train_c2 <- X_train_full

preprocess_c2 <- preProcess(
  X_train_c2,
  method = c("BoxCox", "center", "scale")
)

X_train_c2_bc <- predict(preprocess_c2, X_train_c2)
X_train_c2_final <- spatialSign(X_train_c2_bc)

# Test set for Copy 2
X_test_c2 <- X_test_full[, colnames(X_train_c2), drop = FALSE]
```

```
X_test_c2_bc <- predict(preprocess_c2, X_test_c2)
X_test_c2_final <- spatialSign(X_test_c2_bc)


###################### Transformation Visuals ######################
# Example: Histograms Before vs After Box-Cox on a few variables (Copy 2)

all_vars_c2 <- colnames(X_train_c2)

if (length(all_vars_c2) > 0) {
  for (i in seq(1, min(6, length(all_vars_c2)), by = 3)) {
    vars_chunk <- all_vars_c2[i:min(i+2, length(all_vars_c2))]

    par(mfrow = c(3, 2))
    for (v in vars_chunk) {
      hist(X_train_c2[[v]],
           main = paste(v, "- Before BoxCox (Train)"),
           col  = "tomato", border = "white")

      hist(X_train_c2_bc[[v]],
           main = paste(v, "- After BoxCox (Train)"),
           col  = "steelblue", border = "white")
    }
    par(mfrow = c(1,1))
  }
}
#######################  STEP 7: MODEL BUILDING  ######################

pp_y <- preProcess(
  data.frame(y = y_train),
  method = c("center", "scale")
)

y_train_scaled_df <- predict(pp_y, data.frame(y = y_train))
y_train_scaled    <- y_train_scaled_df$y

# Helper to invert scaling on predictions
inv_scale_y <- function(y_scaled, pp_obj) {
  mu  <- pp_obj$mean["y"]
  sig <- pp_obj$std["y"]
```

```
  return(y_scaled * sig + mu)
}

# TrainControl
ctrl <- trainControl(
  method = "cv",
  number = 10,
  verboseIter = TRUE,
  savePredictions = "final"
)

######################## MODELS USING COPY 1 ##############################
# Linear Regression (OLS), Neural Network, MARS

## 1. Linear Regression (OLS)
set.seed(1)
fit_lm <- train(
  x = X_train_c1_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method = "lm",
  trControl = ctrl
)
print(fit_lm)

## 2. Neural Network (avNNet)
nnet_grid <- expand.grid(
  .size  = c(3, 5, 7, 9),
  .decay = c(0.001, 0.01, 0.1),
  .bag   = FALSE
)

set.seed(1)
fit_nnet <- train(
  x = X_train_c1_final,
  y = y_train_scaled,
  preProcess = c("center", "scale","spatialSign"),
  method  = "avNNet",
  tuneGrid = nnet_grid,
  linout   = TRUE,
```

```
  trace    = FALSE,
  maxit    = 500,
  trControl = ctrl
)

## 3. MARS (earth)
mars_grid <- expand.grid(
  .degree = 1:2,
  .nprune = 2:50
)

set.seed(1)
fit_mars <- train(
  x = X_train_c1_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method   = "earth",
  tuneGrid = mars_grid,
  trControl = ctrl
)

######################## MODELS USING COPY 2 ##############################
# PLS, Ridge, Lasso, Elastic Net, SVM, KNN

## 4. PLS
set.seed(1)
fit_pls <- train(
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method     = "pls",
  tuneLength = 20,
  trControl  = ctrl
)

## 5. Ridge Regression
ridge_grid <- data.frame(.lambda = seq(0, 0.1, length = 15))

set.seed(1)
fit_ridge <- train(
```

```
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method   = "ridge",
  tuneGrid = ridge_grid,
  trControl = ctrl
)

## 6. Lasso
lasso_grid <- data.frame(.fraction = seq(0.05, 1, length = 20))

set.seed(1)
fit_lasso <- train(
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method   = "lasso",
  tuneGrid = lasso_grid,
  trControl = ctrl
)

## 7. Elastic Net (ENet)
enet_grid <- expand.grid(
  .lambda   = c(0, 0.01, 0.1),
  .fraction = seq(0.05, 1, length = 20)
)

set.seed(1)
fit_enet <- train(
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method   = "enet",
  tuneGrid = enet_grid,
  trControl = ctrl
)

## 8. SVM Radial
set.seed(1)
fit_svm <- train(
```

```
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method     = "svmRadial",
  tuneLength = 10,
  trControl  = ctrl
)

## 9. KNN
knn_grid <- data.frame(.k = 1:20)

set.seed(1)
fit_knn <- train(
  x = X_train_c2_final,
  y = y_train_scaled,
  preProcess = c("center", "scale"),
  method   = "knn",
  tuneGrid = knn_grid,
  trControl = ctrl
)

########################  STEP 8: EVALUATION  ##########################
# Collect all models
model_list <- list(LM      = fit_lm,NNet    = fit_nnet,MARS   = fit_mars,PLS    = fit_pls,Ridge  =
fit_ridge,Lasso  = fit_lasso,ENet   = fit_enet,SVM    = fit_svm,KNN    = fit_knn)

#################### TRAINING PERFORMANCE (SCALED Y) ###################

evaluate_model_scaled <- function(model_name, model_object) {
  perf <- getTrainPerf(model_object)
  cat("\nModel:", model_name, "(on SCALED target)\n")
  cat("RMSE (scaled):", round(perf$TrainRMSE, 4), "\n")
  cat("R-sq  (scaled):", round(perf$TrainRsquared, 4), "\n")
  cat("MAE  (scaled):", round(perf$TrainMAE, 4), "\n")
}

cat("\n====================================\n")
cat(" MODEL PERFORMANCE (TRAINING RESAMPLES, SCALED Y)\n")
cat("====================================\n")
```

```
evaluate_model_scaled("Linear Regression (LM)", fit_lm)
evaluate_model_scaled("Neural Network",         fit_nnet)
evaluate_model_scaled("MARS",                   fit_mars)
evaluate_model_scaled("PLS",                    fit_pls)
evaluate_model_scaled("Ridge Regression",       fit_ridge)
evaluate_model_scaled("Lasso",                  fit_lasso)
evaluate_model_scaled("Elastic Net (ENet)",     fit_enet)
evaluate_model_scaled("SVM Radial",             fit_svm)
evaluate_model_scaled("KNN",                    fit_knn)

####################### TEST SET EVALUATION (UNSCALED) ##################

# 1) Predict in SCALED space
# Models trained on Copy 1 use X_test_c1_final
pred_lm_scaled    <- predict(fit_lm,    newdata = X_test_c1_final)
pred_nnet_scaled  <- predict(fit_nnet,  newdata = X_test_c1_final)
pred_mars_scaled  <- predict(fit_mars,  newdata = X_test_c1_final)

# Models trained on Copy 2 use X_test_c2_final
pred_pls_scaled   <- predict(fit_pls,   newdata = X_test_c2_final)
pred_ridge_scaled <- predict(fit_ridge, newdata = X_test_c2_final)
pred_lasso_scaled <- predict(fit_lasso, newdata = X_test_c2_final)
pred_enet_scaled  <- predict(fit_enet,  newdata = X_test_c2_final)
pred_svm_scaled   <- predict(fit_svm,   newdata = X_test_c2_final)
pred_knn_scaled   <- predict(fit_knn,   newdata = X_test_c2_final)

# 2) Back-transform to ORIGINAL energy units
pred_lm    <- inv_scale_y(pred_lm_scaled,    pp_y)
pred_nnet  <- inv_scale_y(pred_nnet_scaled,  pp_y)
pred_mars  <- inv_scale_y(pred_mars_scaled,  pp_y)
pred_pls   <- inv_scale_y(pred_pls_scaled,   pp_y)
pred_ridge <- inv_scale_y(pred_ridge_scaled, pp_y)
pred_lasso <- inv_scale_y(pred_lasso_scaled, pp_y)
pred_enet  <- inv_scale_y(pred_enet_scaled,  pp_y)
pred_svm   <- inv_scale_y(pred_svm_scaled,   pp_y)
pred_knn   <- inv_scale_y(pred_knn_scaled,   pp_y)

# 3) Compute metrics in ORIGINAL units
perf_lm    <- postResample(pred_lm,    y_test)
perf_nnet  <- postResample(pred_nnet,  y_test)
```

```
perf_mars  <- postResample(pred_mars,  y_test)
perf_pls   <- postResample(pred_pls,   y_test)
perf_ridge <- postResample(pred_ridge, y_test)
perf_lasso <- postResample(pred_lasso, y_test)
perf_enet  <- postResample(pred_enet,  y_test)
perf_svm   <- postResample(pred_svm,   y_test)
perf_knn   <- postResample(pred_knn,   y_test)

test_results <- rbind(
  data.frame(Model = "LM",     t(perf_lm)),
  data.frame(Model = "NNet",   t(perf_nnet)),
  data.frame(Model = "MARS",   t(perf_mars)),
  data.frame(Model = "PLS",    t(perf_pls)),
  data.frame(Model = "Ridge",  t(perf_ridge)),
  data.frame(Model = "Lasso",  t(perf_lasso)),
  data.frame(Model = "ENet",   t(perf_enet)),
  data.frame(Model = "SVM",    t(perf_svm)),
  data.frame(Model = "KNN",    t(perf_knn))
)

cat("\n=====================================\n")
cat(" TEST SET PERFORMANCE (ALL MODELS, ORIGINAL SCALE)\n")
cat("=====================================\n")
print(test_results)

########################  TUNING PLOTS (SCALED Y)  ####################

plot(fit_ridge, main = "Ridge Regression: RMSE vs Lambda (Scaled Target)")
plot(fit_pls,   main = "PLS: RMSE vs Number of Components (Scaled Target)")
plot(fit_lasso, main = "Lasso: RMSE vs Fraction (Scaled Target)")
plot(fit_enet,  main = "Elastic Net: RMSE vs Fraction & Lambda (Scaled Target)")
plot(fit_mars,  main = "MARS: RMSE across Degree and nprune (Scaled Target)")
plot(fit_svm,   main = "SVM Radial: RMSE vs Cost/Sigma (Scaled Target)")
plot(fit_knn,   main = "KNN: RMSE vs K (Scaled Target)")
plot(fit_nnet,  main = "Neural Network: RMSE vs Size/Decay (Scaled Target)")

###############  IDENTIFY BEST TWO MODELS (TEST SET)  ##################
test_results_all <- data.frame(
  Model = c("LM", "NNet", "MARS", "PLS", "Ridge", "Lasso", "ENet", "SVM", "KNN"),
  RMSE = c(perf_lm["RMSE"],perf_nnet["RMSE"],perf_mars["RMSE"],perf_pls["RMSE"],
```

```
perf_ridge["RMSE"],perf_lasso["RMSE"],perf_enet["RMSE"],perf_svm["RMSE"],perf_knn["R
MSE"] ),
  Rsquared = c( perf_lm["Rsquared"],perf_nnet["Rsquared"],
perf_mars["Rsquared"],perf_pls["Rsquared"],perf_ridge["Rsquared"],perf_lasso["Rsquared"],
perf_enet["Rsquared"],perf_svm["Rsquared"],perf_knn["Rsquared"  ),
  MAE = c(perf_lm["MAE"],perf_nnet["MAE"],perf_mars["MAE"],
perf_pls["MAE"],perf_ridge["MAE"],perf_lasso["MAE"],perf_enet["MAE"],perf_svm["MAE"],
    perf_knn["MAE"]
  )
)

cat("\n==================================================\n")
cat("  FULL TEST SET PERFORMANCE TABLE (ORIGINAL SCALE)\n")
cat("==================================================\n")
print(test_results_all)

# Rank by RMSE (lower = better)
test_results_sorted <- test_results_all[order(test_results_all$RMSE), ]

cat("\n==================================================\n")
cat("  MODELS RANKED BY TEST RMSE (ORIGINAL SCALE)\n")
cat("==================================================\n")
print(test_results_sorted)

# Select Best Two Models
best_two_models <- test_results_sorted$Model[1:2]

cat("\n==================================================\n")
cat("   BEST TWO MODELS BASED ON TEST RMSE\n")
cat("==================================================\n")
print(best_two_models)

# Store the actual model objects
best_model_1 <- model_list[[ best_two_models[1] ]]
best_model_2 <- model_list[[ best_two_models[2] ]]

cat("\nBest Model 1:", best_two_models[1], "\n")
cat("Best Model 2:", best_two_models[2], "\n")
```

```
# Generate test predictions for top 2 (already in original scale)
if (best_two_models[1] %in% c("LM","NNet","MARS")) {
  pred_best1_scaled <- predict(best_model_1, X_test_c1_final)
} else {
  pred_best1_scaled <- predict(best_model_1, X_test_c2_final)
}
pred_best1 <- inv_scale_y(pred_best1_scaled, pp_y)

if (best_two_models[2] %in% c("LM","NNet","MARS")) {
  pred_best2_scaled <- predict(best_model_2, X_test_c1_final)
} else {
  pred_best2_scaled <- predict(best_model_2, X_test_c2_final)
}
pred_best2 <- inv_scale_y(pred_best2_scaled, pp_y)

perf_best1 <- postResample(pred_best1, y_test)
perf_best2 <- postResample(pred_best2, y_test)

cat("\n==============================================\n")
cat("   TOP TWO MODELS: FINAL TEST PERFORMANCE (ORIGINAL SCALE)\n")
cat("==============================================\n")

cat("\nModel:", best_two_models[1], "\n")
print(perf_best1)

cat("\nModel:", best_two_models[2], "\n")
print(perf_best2)

########################## PRINT ALL MODELS ############################

cat("\n==================== MODEL OBJECTS ====================\n")
for (m in names(model_list)) {
  cat("\n---------------", m, "---------------\n")
  print(model_list[[m]])
}
############ TRAINING PERFORMANCE IN SCALED UNITS (RMSE, MAE, R2) ######

cat("\n\n==================== TRAINING METRICS (SCALED TARGET) ====================\n")
train_metrics <- data.frame()
```

```
for (m in names(model_list)) {
  perf <- getTrainPerf(model_list[[m]])
  train_metrics <- rbind(
    train_metrics,
    data.frame(
      Model = m,
      RMSE_scaled = perf$TrainRMSE,
      MAE_scaled  = perf$TrainMAE,
      R2_scaled   = perf$TrainRsquared
    )
  )
}

print(train_metrics)
############## TEST PERFORMANCE (SCALED TARGET) ##########
cat("\n\n==================== TEST METRICS (SCALED TARGET) ====================\n")
# Predict using scaled target
pred_scaled_list <- list(
  LM    = predict(fit_lm,   X_test_c1_final),
  NNet  = predict(fit_nnet, X_test_c1_final),
  MARS  = predict(fit_mars, X_test_c1_final),
  PLS   = predict(fit_pls,  X_test_c2_final),
  Ridge = predict(fit_ridge, X_test_c2_final),
  Lasso = predict(fit_lasso, X_test_c2_final),
  ENet  = predict(fit_enet, X_test_c2_final),
  SVM   = predict(fit_svm,  X_test_c2_final),
  KNN   = predict(fit_knn,  X_test_c2_final)
)
test_metrics_scaled <- data.frame()
# Scale the y_test for comparison
y_test_scaled <- predict(pp_y, data.frame(y = y_test))$y
for (m in names(pred_scaled_list)) {
  pred <- pred_scaled_list[[m]]
  perf <- postResample(pred, y_test_scaled)
  test_metrics_scaled <- rbind(
    test_metrics_scaled,
    data.frame(
      Model = m,
      RMSE_scaled = perf["RMSE"],
```

```
      MAE_scaled  = perf["MAE"],
      R2_scaled   = perf["Rsquared"]
    )
  )
}

print(test_metrics_scaled)

######################### TOP 10 PREDICTORS #########################

# NNet importance (Copy 1)
nn_imp <- varImp(fit_nnet)$importance
nn_imp$Predictor <- rownames(nn_imp)
top10_nn <- nn_imp[order(-nn_imp$Overall), ][1:10, ]

cat("\nTop 10 Predictors for NNet:\n")
print(top10_nn)

barplot(top10_nn$Overall,
        names.arg = top10_nn$Predictor,
        las = 2,
        cex.names = 0.7,
        main = "Top 10 Predictors - NNET Model",
        col = "darkblue")

# SVM importance (Copy 2)
svm_imp <- varImp(fit_svm)$importance
svm_imp$Predictor <- rownames(svm_imp)
top10_svm <- svm_imp[order(-svm_imp$Overall), ][1:10, ]

cat("\nTop 10 Predictors for SVM:\n")
print(top10_svm)

barplot(top10_svm$Overall,
        names.arg = top10_svm$Predictor,
        las = 2,
        cex.names = 0.7,
        main = "Top 10 Predictors - SVM Model",
        col = "darkred")
```

```
# ---------------- Residuals vs Fitted ---------------- #
par(mfrow = c(1,1))

lm_residuals <- resid(fit_lm$finalModel)
lm_fitted    <- fitted(fit_lm$finalModel)

plot(
  lm_fitted, lm_residuals,
  pch = 19, col = "steelblue",
  xlab = "Fitted Values",
  ylab = "Residuals",
  main = "Residuals vs Fitted (OLS)"
)
abline(h = 0, col = "red", lwd = 2)

# ---------------- Actual vs Fitted ---------------- #
actual <- y_train              # your training target
fitted_vals <- lm_fitted

plot(
  actual, fitted_vals,
  pch = 19, col = "darkgreen",
  xlab = "Actual Values",
  ylab = "Fitted Values",
  main = "Actual vs Fitted (OLS)"
)
abline(0, 1, col = "red", lwd = 2)   # 1-to-1 perfect fit line
ggplot(final_dataset, aes(x = energy)) +
  geom_density(fill = "darkorange", alpha = 0.6, color = "black") +
  labs(
    title = "Density Plot of Energy (After Preprocessing)",
    x = "Energy",
    y = "Density"
  ) +
  theme_minimal()
```